

---

# Table of Contents

|                 |   |
|-----------------|---|
| Problem 1 ..... | 1 |
| Problem 2 ..... | 2 |
| Problem 3 ..... | 5 |
| Functions ..... | 6 |

## Problem 1

%constructs multiple interpolating polynomials, plots  
%function, interpolating polynomials, data, and computes approximations

```
clc  
close all  
clear
```

```
%defines the underlying function  
f = @(x) cos(log(x));
```

```
%creates a vector for x-values of nodes  
nodes = [0.5 1 1.5 2];
```

```
%creates linear interpolating polynomial  
P{1} = @(x) (x-nodes(4))/(nodes(3)-nodes(4))*f(nodes(3))+...  
          (x-nodes(3))/(nodes(4)-nodes(3))*f(nodes(4));
```

```
%creates quadratic interpolating polynomial  
P{2} = @(x) (x-nodes(3)).*(x-nodes(4))/((nodes(2)-nodes(3))*(nodes(2)-  
nodes(4)))*f(nodes(2))+...  
          (x-nodes(2)).*(x-nodes(4))/((nodes(3)-nodes(2))*(nodes(3)-  
nodes(4)))*f(nodes(3))+...  
          (x-nodes(2)).*(x-nodes(3))/((nodes(4)-nodes(2))*(nodes(4)-  
nodes(3)))*f(nodes(4));
```

```
%creates cubic interpolating polynomial  
P{3} = @(x) (x-nodes(2)).*(x-nodes(3)).*(x-nodes(4))/((nodes(1)-  
nodes(2))*(nodes(1)-nodes(3))*(nodes(1)-nodes(4)))*f(nodes(1))+...  
          (x-nodes(1)).*(x-nodes(3)).*(x-nodes(4))/((nodes(2)-nodes(1))*(nodes(2)-  
nodes(3))*(nodes(2)-nodes(4)))*f(nodes(2))+...  
          (x-nodes(1)).*(x-nodes(2)).*(x-nodes(4))/((nodes(3)-nodes(1))*(nodes(3)-  
nodes(2))*(nodes(3)-nodes(4)))*f(nodes(3))+...  
          (x-nodes(1)).*(x-nodes(2)).*(x-nodes(3))/((nodes(4)-nodes(1))*(nodes(4)-  
nodes(2))*(nodes(4)-nodes(3)))*f(nodes(4));
```

```
%creates a vector of x-values for plotting  
xx = 0.4:0.01:2.1;
```

```
%creates a vector of x-values at which approximations are desired  
approximationpoints = [1.75];
```

```
%plots desired output
```

---

```

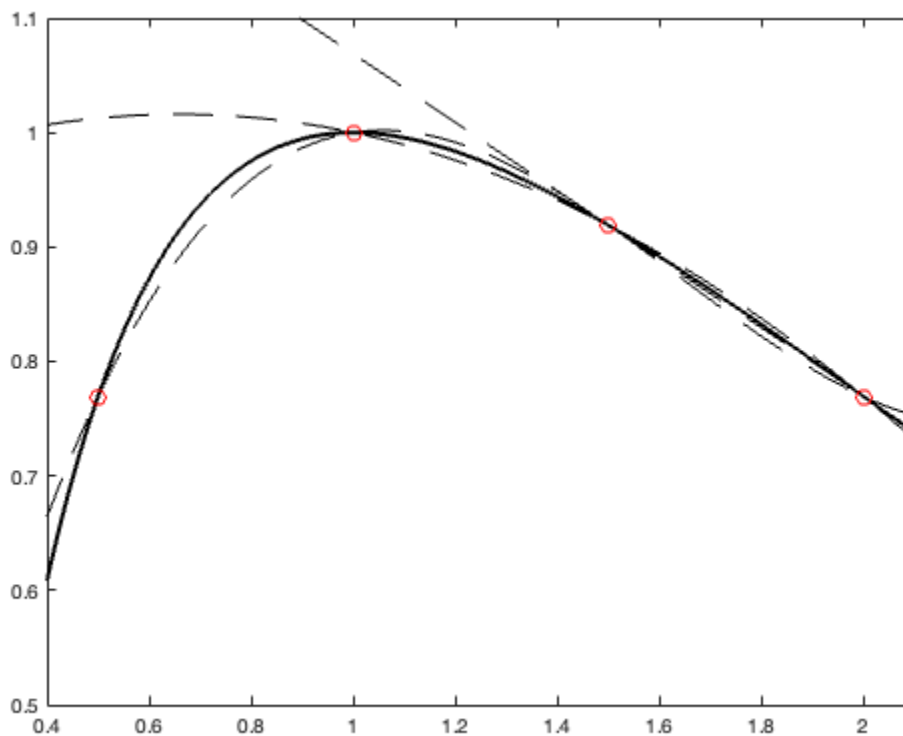
figure(1)
plot(xx, f(xx), 'k', 'LineWidth', 2)
hold on
for i = 1:3
    plot(xx, P{i}(xx), 'k--', 'LineWidth', 1)
    %outputs approximations and absolute error
    for j = 1:1
        fprintf('Approximation at x = %.2f: %8.6f, Absolute
Error: %.6f\n', approximationpoints(j), P{i}(approximationpoints(j)),
abs(f(approximationpoints(j))-P{i}(approximationpoints(j))))
    end
end
plot(nodes, f(nodes), 'ro', 'MarkerSize', 8)
hold off
axis([0.4 2.1 0.5 1.1])

```

```

Approximation at x = 1.75: 0.844079, Absolute Error: 0.003380
Approximation at x = 1.75: 0.852654, Absolute Error: 0.005195
Approximation at x = 1.75: 0.837451, Absolute Error: 0.010008

```



## Problem 2

```

%uses divided differences to compute the coefficients for an interpolating
%polynomial; produces approximations of function values for any vector of x
%values (and can therefore plot the interpolating polynomial easily)

```

---

```

%given x value data
xdata = [-0.10;
         0.00;
         0.20;
         0.30];

% given function value data at corresponding x values
fdata = [17.3000;
         2.0000;
         5.1900;
         1.0000];

%(a) computes the coefficients of the interpolating polynomial using divided
differences

coeffs = DividedDifferences(xdata, fdata);

% (b) produces an approximation for the function value at x = 0.1 and x = 0.4

x = [0.1 0.4];

approx = DividedDifferences_val(xdata, fdata,x);
fprintf('Approximations generated using %i data points\n', length(xdata))
for i = 1:length(x)
    fprintf('Approximate function value at %f is %.5f\n', x(i), approx(i))
end
fprintf('\n')

% (c) produces approximations for a vector of x values to graph the
interpolating polynomial

%vector of x values for which we will generate approximations
x = -0.2:0.01:0.5;

f = DividedDifferences_val(xdata, fdata,x);
figure(2)
plot(xdata,fdata,'o r',x,f,'-k')

% (d) add a data point and update the plot from (a)

%given x value data
xdata_1 = [-0.10;
           0.00;
           0.20;
           0.30;
           0.05];

%given function value data at corresponding x values
fdata_1 = [17.3000;
           2.0000;
           5.1900;
           1.0000;
           3.1250];

```

---

---

```

x = [0.1 0.4];

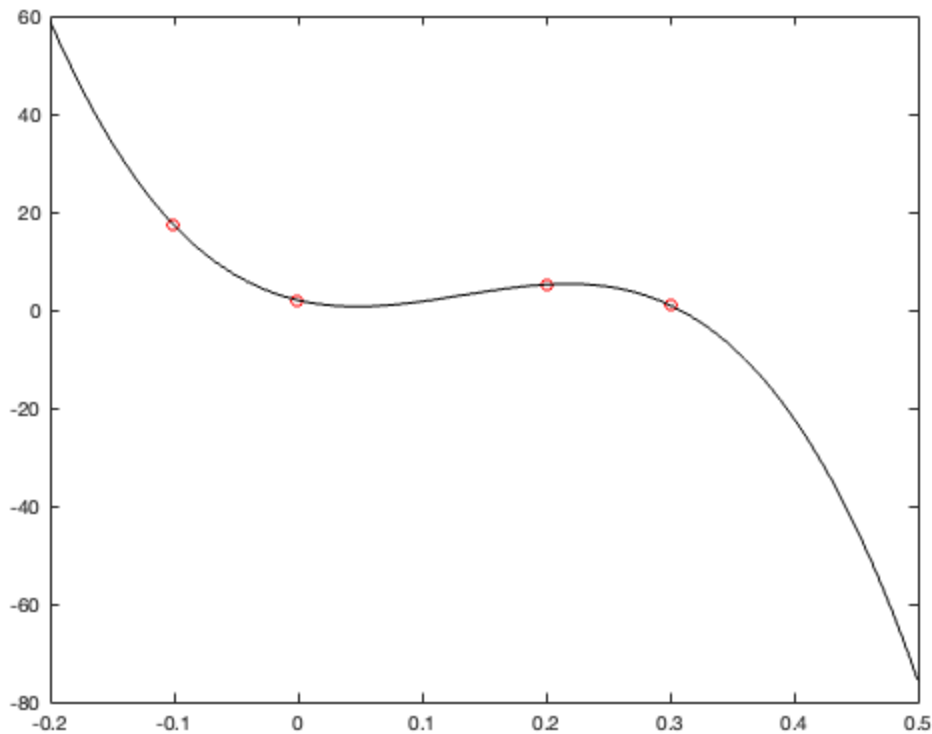
approx_2 = DividedDifferences_val(xdata_1, fdata_1,x);
fprintf('Approximations generated using %i data points\n', length(xdata))
for i = 1:length(x)
    fprintf('Approximate function value at %f is %.5f\n', x(i), approx_2(i))
end
fprintf('\n')

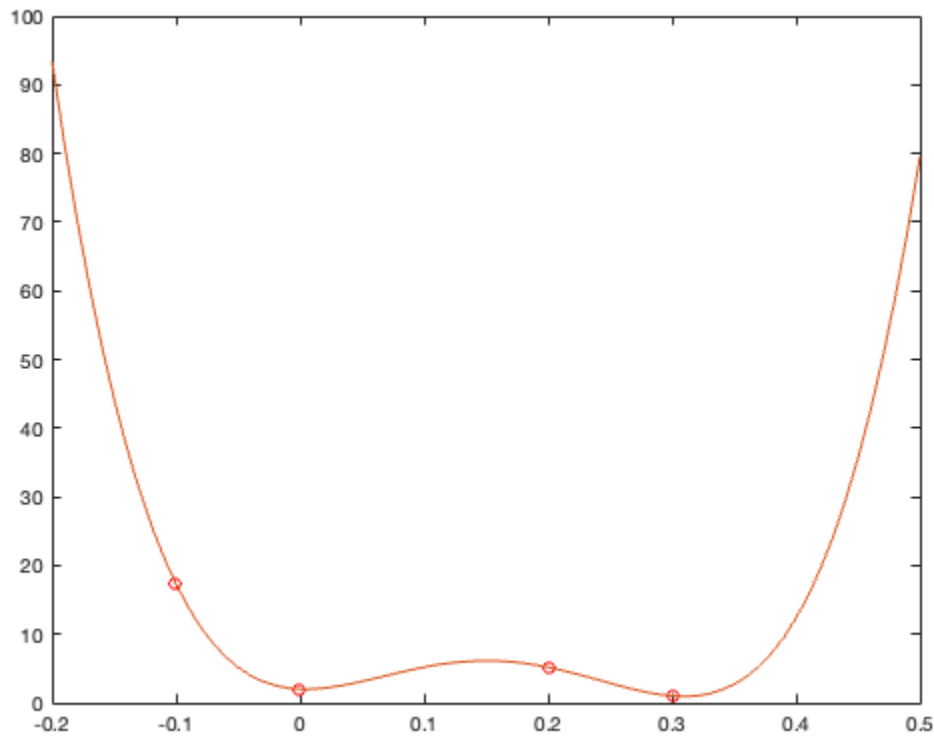
% (e)
%vector of x values for which we will generate approximations
x = -0.2:0.01:0.5;
f_1 = DividedDifferences_val(xdata_1, fdata_1,x);
figure(3)
plot(xdata,fdata,'o r',x,f_1)

Approximations generated using 4 data points
Approximate function value at 0.100000 is 1.74333
Approximate function value at 0.400000 is -22.16667

Approximations generated using 4 data points
Approximate function value at 0.100000 is 5.19222
Approximate function value at 0.400000 is 12.32222

```





## Problem 3

`%finds the natural cubic spline used to predict race time`

`%creates vectors for given data: x for x values and a for function values`

```
x = [0 0.25 0.5 1 1.25];
```

```
a = [0 23.04 47.37 97.45 123.66];
```

`%calls function to compute unknown coefficients`

```
[n, b, c, d] = cubicspline(x, a);
```

`%computes the time at the three-quarter mile mark`

```
xx = 0.75;
```

```
i = 3;
```

```
time = a(i)+b(i)*(xx-x(i))+c(i)*(xx-x(i)).^2+d(i)*(xx-x(i)).^3;
```

```
fprintf('Predicted race time: %i:%.2f\n\n', floor(time/60), mod(time, 60))
```

`%computes the derivative at a specific point`

```
xx = 0;
```

```
i = 1;
```

```
speed = 1/(b(i)+2*c(i)*(xx-x(i))+3*d(i)*(xx-x(i)).^2);
```

```
fprintf('Predicted speed at the start of the race: %.2f mi/hr\n\n',  
speed*3600)
```

`%computes the derivative at a specific point`

---

```

xx = x(end);
i = n;
speed = 1/(b(i)+2*c(i)*(xx-x(i))+3*d(i)*(xx-x(i)).^2);
fprintf('Predicted speed at the end of the race: %.2f mi/hr\n\n', speed*3600)

```

## Functions

```

function [n, b, c, d] = cubicspline(x, a)
    %finds largest index (n+1 nodes)
    n = length(x)-1;

    %creates vector of subinterval lengths
    h = x(2:end)-x(1:end-1);

    %creates empty coefficient matrix
    A = zeros(n+1, n+1);

    %updates entries on main diagonal, super diagonal and sub diagonal of
    %the coefficient matrix
    A(2:end-1, 2:end-1) = diag(2*(h(1:end - 1) + h(2:end)))+diag(h(2:end-1),
1)+diag(h(2:end-1), -1);

    %updates entries in the first and last rows of the matrix
    A(1,1) = 1;
    A(2,1) = h(1);
    A(end-1,end) = h(end);
    A(end,end) = 1;

    %creates empty right-hand side vector
    B = zeros(n+1,1);

    %updates entries in the right-hand side vector
    B(2:end-1) = 3./h(2:end).*(a(3:end)-a(2:end-1))-3./
h(1:end-1).*(a(2:end-1)-a(1:end-2));

    %solves for c values
    c = (A\B)';

    %solves for b and d values (using c values)
    b = (a(2:end)-a(1:end-1))./h-h/3.*(2*c(1:end-1)+c(2:end));
    d = (c(2:end)-c(1:end-1))./(3*h);
end

% function cubicsplineplot(n, x, a, b, c, d, exact)
%     figure
%     hold on
%     %individual iteration plots cubic function on a single subinterval
%     for i = 1:n
%         xx = x(i):(x(i+1)-x(i))/100:x(i+1);
%         plot(xx, a(i)+b(i)*(xx-x(i))+c(i)*(xx-x(i)).^2+d(i)*(xx-x(i)).^3,
'b', 'LineWidth', 1)
%     end
%     %plots exact solution on the entire interval

```

---

```

%     xxinterval = x(1):(x(end)-x(1))/100:x(end);
%     plot(xxinterval, exact(xxinterval), 'k', 'LineWidth', 1)
%     hold off
% end

function coeffs = DividedDifferences(xdata, fdata)
%computes the coefficients of an interpolating polynomial given function
%values fdata at x values xdata

%INPUT
% xdata: given x value data
% fdata: given function value data at corresponding x values

%OUTPUT
% coeffs: vector of coefficients a_0, a_1, a_2,..., a_n for interpolating
%         polynomial

%creates a table to compute and store divided differences values
table = zeros(length(xdata), length(xdata));

%uses the function value data as the first column of table
table(:, 1) = fdata;

%generates lower triangular matrix table iteratively, working column by
%column
for j = 2:length(xdata)
    for i = j:length(xdata)
        %implements divided differences
        table(i,j) = (table(i,j-1)-table(i-1,j-1))/(xdata(i)-xdata(i-
j+1));
    end
end

%strips only the diagonal elements from table and creates a vector of
%coefficients
coeffs = diag(table);
end

function approx_val = DividedDifferences_val(xdata, fdata, x_val)
%computes the coefficients of an interpolating polynomial given function
%values fdata at x values xdata

%INPUT
% xdata: given x value data
% fdata: given function value data at corresponding x values

%OUTPUT
% coeffs: vector of coefficients a_0, a_1, a_2,..., a_n for interpolating
%         polynomial

%creates a table to compute and store divided differences values
table = zeros(length(xdata), length(xdata));

%uses the function value data as the first column of table

```

---

---

```

table(:, 1) = fdata;

%generates lower triangular matrix table iteratively, working column by
%column
for j = 2:length(xdata)
    for i = j:length(xdata)
        %implements divided differences
        table(i,j) = (table(i,j-1)-table(i-1,j-1))/(xdata(i)-xdata(i-
j+1));
    end
end

%strips only the diagonal elements from table and creates a vector of
%coefficients
coeffs = diag(table);

approx_val = zeros(1, length(x_val));

for i = 1:length(x_val)
    x = x_val(i);
    xdiff = x-xdata;

    %creates a vector to store the products for terms with coeff
a_1,...,a_n
    products = ones(1, length(coeffs));

    %computes the products required for each term a_1,...,a_n (since the
first
    %term is simply a_0, the first element is not updated)
    for j = 2:length(coeffs)
        products(j) = prod(xdiff(1:j-1));
    end

    approx_val(1,i) = products*coeffs;
end
end

```

*Predicted race time: 1:12.12*

*Predicted speed at the start of the race: 39.62 mi/hr*

*Predicted speed at the end of the race: 34.08 mi/hr*

*Published with MATLAB® R2024b*